INFORME

I. PORTADA

Tema:	Árboles en C++
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Nivel - Paralelo
Alumno:	Bravo Laica Geremy Alexis
Asignatura:	Estructura de Datos
Docente:	Ing. José Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

Implementar en C++ un Árbol Binario de Búsqueda — BST para organizar empleados de una empresa usando un código numérico como clave. El ejercicio se relaciona con la diapositiva del organigrama empresarial, donde se identifican raíz, niveles, nodos internos y hojas.

2.2 Modalidad

Presencial.

2.3 Tiempo de duración

Presenciales: 8
No presenciales: 0

2.4 Instrucciones

Una empresa necesita registrar empleados en una estructura jerárquica usando un árbol binario de búsqueda.

Cada empleado tendrá:

- Código
- Nombre
- Cargo

El árbol debe permitir:

Insertar empleados.
Buscar un empleado por código.
Mostrar recorrido inorden.
Mostrar recorrido preorden.
Mostrar recorrido postorden.
Calcular altura del árbol.
Identificar hojas.
Mostrar la raíz del árbol.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

-

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

☐Plataformas educativas
☐Simuladores y laboratorios virtuales

☐Aplicaciones educativas
☐Recursos audiovisuales
☐Gamificación
☐Inteligencia Artificial
Otros (Especifique): _____

2.6 Actividades por desarrollar

1. RolResponsabilidad
2. Líder Coordina el avance y revisa el repositorio
3. Programador Implementa estructura Nodo y función insertar
4. Programador Implementa búsqueda y recorridos
5. Programador Implementa altura y hojas
6. Documentador Crea README, capturas y explicación
7. Expositor Presenta funcionamiento y conclusiones.

Datos de prueba sugeridos

Código	Nombre	Cargo
50	Empresa UTA	Raíz
30	Gerente Ventas	Nodo interno
70	Gerente Finanzas	Nodo interno
20	Emp 1	Hoja
40	Emp 2	Hoja
60	Emp 3	Hoja
80	Emp 4	Hoja

Implementación en GitHub paso a paso
Paso 1: Crear repositorio

Nombre sugerido:

arbol-bst-empresa-cpp

Debe contener:

README.md
src/main.cpp
capturas/
Paso 2: Clonar el repositorio
git clone https://github.com/usuario/arbol-bst-empresa-cpp.git
cd arbol-bst-empresa-cpp
Paso 3: Crear carpetas
mkdir src
mkdir capturas
Paso 4: Crear archivo principal
cd src
touch main.cpp

Pegar el código C++ en main.cpp.

Paso 5: Compilar
g++ main.cpp -o arbol
Paso 6: Ejecutar
./arbol

En Windows:

arbol.exe
Paso 7: Subir a GitHub
git add .
git commit -m "Implementacion de arbol BST empresarial en C++"
git push origin main
7. Contenido mínimo del README
Árbol BST Empresarial en C++

Integrantes
- Nombre 1
- Nombre 2
- Nombre 3
- Nombre 4

Objetivo
Implementar un árbol binario de búsqueda en C++ para organizar empleados de una empresa.

Funcionalidades
- Insertar empleados
- Buscar empleados
- Mostrar raíz
- Recorridos inorden, preorden y postorden
- Calcular altura
- Mostrar nodos hoja

Capturas
Agregar capturas de:
1. Menú principal
2. Inserción de empleados
3. Búsqueda
4. Recorridos
5. Altura y hojas

8. Evidencia que debe entregar el grupo
- Link del repositorio GitHub.
- Código fuente en C++.
- README completo.
- Capturas de ejecución.
- Explicación breve de raíz, nodo interno, hoja, nivel y altura.
- Participación de cada integrante mediante commits.

2.7 Resultados obtenidos

Árbol Binario de Búsqueda
Un Árbol Binario de Búsqueda (BST) es una estructura de datos jerárquica donde cada nodo tiene como máximo dos hijos. La propiedad fundamental es que todos los nodos del subárbol izquierdo tienen valores menores al nodo padre, y todos los del subárbol derecho tienen valores mayores.
Conceptos Clave
- Raiz: nodo superior del árbol, sin padre.
- Nodo interno: nodo que tiene al menos un hijo.
- Hoja: nodo sin hijos.

- Altura: numero de niveles desde la raiz hasta la hoja mas lejana.
- Nivel: profundidad de un nodo dentro del arbol.

Recorridos

- Inorden (Izquierda - Raiz - Derecha): produce los elementos ordenados de menor a mayor.
- Preorden (Raiz - Izquierda - Derecha): util para copiar o serializar el arbol.
- Postorden (Izquierda - Derecha - Raiz): util para liberar memoria o evaluar expresiones.

Descripción del Programa

El programa implementa un menú interactivo en consola con las siguientes funcionalidades:

- Insertar empleado (código, nombre, cargo)
- Buscar empleado por código
- Mostrar la raíz del árbol
- Recorrido inorden
- Recorrido preorden
- Recorrido postorden
- Calcular y mostrar la altura del arbol
- Identificar y mostrar los nodos hoja

Estructura del Código

Struct Empleado

Almacena los datos de cada empleado: código (int), nombre (string) y cargo (string).

Struct Nodo

Representa cada nodo del árbol. Contiene un objeto Empleado y punteros al hijo izquierdo y derecho. El constructor inicializa ambos punteros en nullptr.

Clase ArbolBST

Contiene el puntero a la raíz y los métodos privados recursivos para insertar, buscar, recorrer, calcular altura y mostrar hojas. Los métodos públicos son la interfaz para el main.

Capturas

1. Menú principal

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion:
```

2. Inserción de empleados

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 1
Codigo: 50
Nombre: Empresa UTA
Cargo: Raíz
```

3. Búsqueda

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 2
Ingrese codigo a buscar: 70

Empleado encontrado:
Codigo: 70 | Nombre: Gerente Finanzas | Cargo: Nodo interno
```

4. Recorridos
   - Mostrar recorrido inorden.

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas       4
0. Salir
Recorrido Inorden:ion:
Codigo: 20 | Nombre: Emp  | Cargo: Hoja
Codigo: 30 | Nombre: Gerente Ventas | Cargo: Nodo interno
Codigo: 40 | Nombre: Emp 2 | Cargo: Hoja
Codigo: 50 | Nombre: Empresa UTA        | Cargo: Raíz
Codigo: 60 | Nombre: Emp 3 | Cargo: Hoja
Codigo: 70 | Nombre: Gerente Finanzas | Cargo: Nodo interno
Codigo: 80 | Nombre: Emp 4 | Cargo: Hoja
```

   - Mostrar recorrido preorden.

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 5

Recorrido Preorden:
Codigo: 50 | Nombre: Empresa UTA         | Cargo: Raíz
Codigo: 30 | Nombre: Gerente Ventas | Cargo: Nodo interno
Codigo: 20 | Nombre: Emp  | Cargo: Hoja
Codigo: 40 | Nombre: Emp 2 | Cargo: Hoja
Codigo: 70 | Nombre: Gerente Finanzas | Cargo: Nodo interno
Codigo: 60 | Nombre: Emp 3 | Cargo: Hoja
Codigo: 80 | Nombre: Emp 4 | Cargo: Hoja
```

- Mostrar recorrido postorden.

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 6

Recorrido Postorden:
Codigo: 20 | Nombre: Emp  | Cargo: Hoja
Codigo: 40 | Nombre: Emp 2 | Cargo: Hoja
Codigo: 30 | Nombre: Gerente Ventas | Cargo: Nodo interno
Codigo: 60 | Nombre: Emp 3 | Cargo: Hoja
Codigo: 80 | Nombre: Emp 4 | Cargo: Hoja
Codigo: 70 | Nombre: Gerente Finanzas | Cargo: Nodo interno
Codigo: 50 | Nombre: Empresa UTA         | Cargo: Raíz
```

5. Altura y hoja.

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 7

Altura del arbol: 3
```

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 8

Nodos hoja:
Codigo: 20 | Nombre: Emp  | Cargo: Hoja
Codigo: 40 | Nombre: Emp 2 | Cargo: Hoja
Codigo: 60 | Nombre: Emp 3 | Cargo: Hoja
Codigo: 80 | Nombre: Emp 4 | Cargo: Hoja
```

2.8 Habilidades blandas empleadas en la práctica

☐ Liderazgo
☐ Trabajo en equipo
☐ Comunicación asertiva
☐ La empatía
☒ Pensamiento crítico
☐ Flexibilidad
☐ La resolución de conflictos
☒ Adaptabilidad
☒ Responsabilidad

2.9 Conclusiones

- El Árbol Binario de Búsqueda permite organizar empleados de forma jerárquica y realizar búsquedas eficientes con complejidad O(log n) en el caso promedio.
- La implementación recursiva simplifica el código de inserción, búsqueda y recorridos.
- Los tres tipos de recorrido permiten obtener la información en distintos ordenes según la necesidad.
- El cálculo de altura y la identificación de hojas son útiles para analizar el estado y balance del árbol.
- Esta estructura de datos es aplicable a sistemas reales de gestión de empleados, catálogos y directorios.

2.10 Anexos

https://github.com/GeremyBravo/arbol-bst-empresa-cpp.git